

laSalle

UNIVERSITAT RAMON LLULL

**Escola Tècnica Superior d'Enginyeria
Electrònica i Informàtica La Salle**

Trabajo Final de Máster

MÁSTER BIG DATA

Evaluación Macroeconómica

Estudiantes

Lucas Alexis Morales Bahamondes
Ismael Angel Fernandez Paredes

Profesor Ponente

Mentor Externo

**ACTA DEL EXAMEN
DEL TRABAJO FINAL DE
MÁSTER**

Reunido el Tribunal calificador en la fecha indicada, los estudiantes

D. Lucas Alexis Morales Bahamondes e Ismael Angel Fernandez Paredes

expusieron su Trabajo Final de Máster, titulado:

Evaluación Macroeconómica

Acabada la exposición y contestadas por parte de los estudiantes las objeciones formuladas por los Sres. miembros del tribunal, éste valoró dicho Trabajo con la calificación de

Barcelona,

VOCAL DEL TRIBUNAL

VOCAL DEL TRIBUNAL

PRESIDENTE DEL TRIBUNAL

Índice general

Abstract	1
Agradecimientos	2
Acrónimos	3
1. Introducción	4
2. Contexto, Objetivos y Validación	5
2.1. Contexto y Motivación	5
2.2. Objetivos del Proyecto	6
2.3. Validación: Preguntas de investigación respondidas	7
2.3.1. Bloque A: Inflación y Coste de Vida	7
2.3.2. Bloque B: Costo de Vida y Vivienda	8
2.3.3. Economía Local y Sectores Productivos	8
2.3.4. Bloque D: Turismo y Presión Urbana	9
2.3.5. Bloque E: Energía, Transporte y Vida Diaria	10
2.3.6. Bloque F: Comparación General y Decisión Ciudadana	10
3. Arquitectura de Datos Propuesta	12
3.1. Visión general	12
3.2. Arquitectura AWS actualizada	13
3.3. Justificación del almacenamiento SQL para consultas conversacionales	14
3.4. Flujo de consulta desde la página web	14
3.5. Decisiones de diseño	15
3.6. Capas de procesamiento	16
3.7. Estrategia jerárquica de fuentes	16
3.8. Relación con el entorno local	17

3.9. Gestión de errores y reproceso	17
3.10. Gobernanza y calidad del dato	18
4. Modelo y Flujo de Datos	20
4.1. Principios del modelo de datos	20
4.2. Modelo dimensional actualizado	20
4.3. Catálogos maestros	22
4.4. Territorios y niveles de análisis	23
4.5. Ingesta y datos brutos	23
4.6. Metadatos y validación	23
4.7. Datos curados y normalización	24
4.8. Cálculo de scores	24
4.9. Histórico, auditoría y reproducibilidad	24
4.10. Relación entre flujo lógico y arquitectura AWS	25
5. Capítulo 2: Creación del Entorno Local de Desarrollo y Validación	27
6. Estructura General del Proyecto	28
6.1. Creación de los Conectores API	30
6.2. Pipeline reproducible de limpieza y carga SQL	33
6.2.1. Scripts implementados	34
6.2.2. Resumen cuantitativo por fase	36
6.2.3. Resultados de recolección e inventario Bronze	36
6.2.4. Capa Silver: datos limpios y seleccionados	39
6.2.5. Capa Gold y base SQLite	40
6.2.6. Relación con la página web y el chat	43
6.2.7. Alcance SQL consolidado y limitaciones actuales	45
7. Conclusiones	47
Referencias	48

Abstract

¿Qué problema aborda el proyecto? ISEU+ aborda la dificultad de analizar y comparar la situación económica de distintas localidades de España debido a la dispersión de los datos públicos. Aunque existen fuentes oficiales con información sobre empleo, coste de vida, vivienda, turismo, energía o actividad económica, estos datos se encuentran repartidos entre distintos organismos, formatos y niveles territoriales, lo que dificulta su integración y análisis conjunto.

¿Cuál es el objetivo del trabajo? El objetivo del trabajo es diseñar una plataforma de análisis económico urbano basada en Big Data, capaz de recopilar, limpiar, normalizar y estructurar datos oficiales para convertirlos en indicadores comparables entre localidades españolas. El sistema se plantea como una base de datos trazable y preparada para su posterior ejecución en una arquitectura AWS.

¿Qué resultados se esperan obtener? El proyecto espera obtener una estructura de datos que permita comparar ciudades y municipios mediante indicadores relacionados con empleo, coste de vida, vivienda, turismo, energía y actividad económica. El modelo está diseñado desde el inicio para comparar localidades de toda España y responder preguntas ciudadanas como dónde hay más oportunidades laborales, qué ciudades presentan mayor presión turística o cómo varía el coste de vida entre territorios.

¿Qué conclusiones aporta el proyecto? ISEU+ demuestra la importancia de organizar y transformar datos públicos dispersos en información útil para el análisis territorial. El valor principal del proyecto no se encuentra en una interfaz concreta, sino en la construcción de una plataforma de datos capaz de apoyar comparaciones entre localidades y facilitar una toma de decisiones más informada.

Agradecimientos

Este capítulo es opcional.

Acrónimos

FPU: Floating Point Unit.

ICT: Infraestructura Común de Telecomunicaciones.

OFDM: Orthogonal Frequency Division Multiplexing.

TFG: Trabajo Final de Grado.

En este capítulo se listan los acrónimos que aparecen en el documento, por orden alfabético. Ya no se desplegarán de nuevo en el resto del documento. Si los acrónimos provienen de palabras escritas en un idioma diferente al del documento, se escribirá en cursiva.

Capítulo 1

Introducción

Este Trabajo Final de Máster presenta el desarrollo de ISEU+ (Índice de Salud Económica Urbana), una plataforma de análisis económico urbano orientada a evaluar y comparar la situación de distintas localidades de España a partir de datos oficiales. El proyecto se plantea con alcance nacional y está pensado para comparar ciudades y municipios de España, como Madrid, Barcelona, Valencia, Sevilla, Málaga, Bilbao, Zaragoza u otras localidades con características económicas y urbanas diferentes.

ISEU+ busca integrar información socioeconómica dispersa en diferentes fuentes públicas para construir una visión más clara de la salud económica urbana. El sistema se apoya en indicadores relacionados con empleo, coste de vida, dinamismo económico, vivienda, turismo, accesibilidad, bienestar y entorno empresarial, con el objetivo de facilitar comparaciones entre territorios y responder preguntas ciudadanas basadas en datos.

La fase actual del proyecto consiste en desarrollar y validar el sistema en un entorno local. En esta etapa se prioriza la identificación de fuentes fiables, la creación de conectores, la limpieza de datos, la estructuración de variables y la validación del modelo analítico. Una vez comprobado que el sistema funciona correctamente y que los datos son consistentes, el proyecto migrará a una arquitectura en la nube, donde los procesos podrán automatizarse, escalarse y desplegarse como producto web.

El objetivo principal no es únicamente construir una interfaz o un chatbot, sino desarrollar una base de datos trazable y una metodología de análisis que permita comparar ciudades de forma responsable. La interfaz conversacional se plantea como una capa de acceso para que el usuario pueda realizar preguntas en lenguaje natural, mientras que el núcleo del trabajo se encuentra en el modelo de datos, la calidad de las fuentes y la capacidad del sistema para generar respuestas explicables.

Capítulo 2

Contexto, Objetivos y Validación

2.1. Contexto y Motivación

ISEU+ nace de la necesidad de recopilar, integrar y analizar datos socioeconómicos de distintas localidades de España. Aunque existen numerosas fuentes oficiales con información pública, los datos se encuentran dispersos entre organismos, formatos y niveles territoriales diferentes. Esta fragmentación dificulta realizar comparaciones entre ciudades y extraer conclusiones útiles sobre empleo, coste de vida, vivienda, turismo, energía o actividad económica.

Desde la perspectiva de Big Data, el reto principal del proyecto consiste en construir un flujo completo de tratamiento de datos: identificación de fuentes, extracción mediante APIs o descargas oficiales, limpieza, normalización, almacenamiento estructurado y preparación para análisis comparativo. El objetivo es transformar datos públicos heterogéneos en una base de información coherente, trazable y útil para evaluar la salud económica urbana.

Fuente	Tipo de información	Uso dentro de ISEU+
INE	IPC, empleo, población e indicadores económicos	Comparación territorial y análisis socioeconómico
Idescat	Datos económicos y demográficos autonómicos	Datos de referencia para Cataluña y contraste con fuentes estatales
REE	Precios eléctricos y demanda energética	Análisis del impacto energético
MITMA / MI-VAU	Vivienda, transporte y datos territoriales	Evaluación de vivienda y accesibilidad
SEPE: Seguridad Social	Paro registrado, afiliación y empleo	Análisis del mercado laboral

Fuente	Tipo de información	Uso dentro de ISEU+
Open Data municipales	Datos locales publicados por ayuntamientos	Incorporación de indicadores urbanos específicos

El proyecto se plantea con una arquitectura orientada a AWS, donde los procesos de extracción, almacenamiento, transformación y análisis podrán automatizarse y escalarse. De este modo, ISEU+ se concibe como una plataforma de datos capaz de integrar información pública de distintas fuentes y convertirla en indicadores comparables para el análisis económico urbano.

2.2. Objetivos del Proyecto

El objetivo general de ISEU+ es desarrollar una plataforma de análisis económico urbano basada en datos oficiales, capaz de recopilar, integrar y comparar información socioeconómica de distintas localidades de España. El proyecto busca transformar datos públicos dispersos en indicadores estructurados que permitan analizar empleo, coste de vida, vivienda, energía, turismo, actividad económica y otros factores relacionados con la salud económica urbana.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

Identificar y seleccionar fuentes oficiales relevantes para el análisis socioeconómico de localidades españolas.

Diseñar procesos de extracción de datos mediante APIs, descargas oficiales u otros mecanismos documentados.

Limpiar, normalizar y estructurar datos heterogéneos en un modelo común preparado para el análisis comparativo.

Construir un pipeline de datos orientado a su posterior ejecución en una arquitectura AWS.

Definir indicadores que permitan comparar ciudades en dimensiones como empleo, coste de vida, vivienda, turismo, energía y actividad económica.

Validar el sistema mediante preguntas ciudadanas que representen consultas reales sobre distintas localidades de España.

El alcance del proyecto se centra en el diseño y desarrollo de una base de datos analítica para comparar territorios. El modelo se plantea para trabajar con localidades españolas y permitir comparativas entre ciudades con características económicas y urbanas diferentes.

2.3. Validación: Preguntas de investigación respondidas

Para validar la utilidad práctica de ISEU+, se han definido 30 preguntas representativas que el sistema debería ser capaz de responder una vez consolidada la capa de datos. Estas preguntas no se presentan como resultados definitivos ya extraídos, sino como una guía de validación funcional y metodológica. El objetivo es comprobar si las fuentes seleccionadas, los conectores locales, el modelo de datos y la futura capa de consulta permiten transformar datos oficiales dispersos en respuestas claras, trazables y comprensibles.

En esta fase del proyecto, las preguntas sirven para orientar la investigación de datos: ayudan a identificar qué variables son necesarias, qué fuentes deben priorizarse, qué indicadores requieren limpieza o normalización y qué limitaciones pueden aparecer por falta de granularidad territorial o frecuencia de actualización. Por tanto, cada respuesta esperada describe cómo debería contestar el sistema, qué tipo de dato debería utilizar y qué fuente sería la más adecuada para sostener la respuesta.

2.3.1. Bloque A: Inflación y Coste de Vida

Nº	Pregunta	Respuesta esperada
1	¿Dónde hay más oportunidades de empleo, en Madrid, Barcelona, Valencia o Sevilla?	El sistema debería comparar indicadores laborales equivalentes entre ciudades o provincias, como tasa de paro, afiliación, ocupación o evolución reciente del empleo. La respuesta debería explicar qué territorio presenta mejores señales laborales y advertir si algún dato solo está disponible a nivel provincial o autonómico.
2	¿En qué ciudad española está bajando más el paro últimamente?	El sistema debería analizar la evolución reciente del paro en varias ciudades o provincias y ordenar los territorios según la reducción observada. También debería diferenciar entre mejora puntual y tendencia sostenida.
3	¿Dónde parece más fácil encontrar trabajo para vivir: Málaga, Zaragoza, Bilbao o Alicante?	El sistema debería comparar empleo, paro y, si es posible, coste de vida o vivienda, ya que una ciudad con más empleo no siempre ofrece mejores condiciones reales para vivir.
4	¿Qué ciudades tienen un mercado laboral más estable durante todo el año?	El sistema debería detectar si el empleo depende mucho de temporadas, como verano, turismo o campañas agrícolas, y señalar qué ciudades muestran menos variación estacional.

Nº	Pregunta	Respuesta esperada
5	¿Qué zonas dependen más del turismo para generar empleo?	El sistema debería comparar empleo en hostelería, turismo o servicios relacionados entre ciudades como Palma, Benidorm, Málaga, Barcelona, Sevilla o Las Palmas, explicando si existe dependencia sectorial.

2.3.2. Bloque B: Costo de Vida y Vivienda

Nº	Pregunta	Respuesta esperada
6	¿Dónde sale más caro vivir, en Barcelona, Madrid, San Sebastián, Málaga o Valencia?	El sistema debería comparar indicadores de coste de vida disponibles, como IPC territorial, alquiler, precio de vivienda, IBI u otros datos relacionados. La respuesta debería explicar qué ciudad parece más cara y qué indicador pesa más en esa conclusión.
7	¿En qué ciudad se ha encarecido más la vivienda en los últimos años?	El sistema debería analizar la evolución del precio de compra o alquiler en distintas ciudades y señalar dónde el aumento ha sido más fuerte.
8	¿Dónde cuesta más alquilar en relación con los ingresos, en Madrid, Barcelona, Palma o Málaga?	El sistema debería relacionar precios de alquiler con renta o salario disponible, indicando dónde el esfuerzo económico para alquilar parece más alto.
9	¿Qué ciudades parecen más asequibles para vivir si tengo un salario medio?	El sistema debería combinar coste de vivienda, salarios o renta disponible y otros costes básicos para ofrecer una comparación orientativa, no una recomendación absoluta.
10	¿La compra del supermercado está subiendo igual en todas las zonas de España?	El sistema debería comparar la evolución del IPC general y de alimentos por comunidades o provincias, explicando si la inflación afecta de forma desigual al territorio.

2.3.3. Economía Local y Sectores Productivos

Nº	Pregunta	Respuesta esperada
11	¿De qué vive principalmente cada ciudad: turismo, industria, comercio, servicios o construcción?	El sistema debería identificar el peso de los sectores económicos en diferentes ciudades o territorios y explicar cuál es la actividad dominante.
12	¿Qué ciudad depende más del turismo: Palma, Barcelona, Benidorm, Málaga o Las Palmas?	El sistema debería comparar indicadores turísticos como plazas hoteleras, peso de hostelería, visitantes o empleo turístico, señalando si existe una concentración elevada.
13	¿Qué ciudades tienen más peso industrial, Bilbao, Zaragoza, Vigo, Valladolid o Valencia?	El sistema debería comparar el peso de la industria mediante VAB sectorial, empleo industrial o indicadores disponibles, explicando qué territorios tienen una estructura más industrial.
14	¿Dónde tiene más peso el comercio, en Sevilla, Valencia, Madrid o Alicante?	El sistema debería comparar indicadores de actividad comercial y explicar qué ciudades muestran mayor dependencia o dinamismo del comercio.
15	¿Qué ciudades tienen una economía más diversificada?	El sistema debería analizar la distribución sectorial y señalar qué territorios dependen menos de un único sector económico.

2.3.4. Bloque D: Turismo y Presión Urbana

Nº	Pregunta	Respuesta esperada
16	¿En qué ciudades hay más presión turística sobre los residentes?	El sistema debería calcular ratios como plazas hoteleras por habitante, visitantes por residente o peso del alojamiento turístico, comparando ciudades como Palma, Barcelona, Málaga, Sevilla, Granada, Valencia o San Sebastián.
17	¿Hay demasiados hoteles en relación con la población local en algunas ciudades?	El sistema debería relacionar plazas hoteleras y población residente para detectar territorios con alta concentración turística.
18	¿Qué ciudades reciben más turismo en comparación con su tamaño?	El sistema debería comparar visitantes, pernoctaciones o plazas turísticas en relación con población, no solo en valores absolutos.

Nº	Pregunta	Respuesta esperada
19	¿El turismo genera más riqueza que otros sectores en ciudades como Palma, Málaga o Barcelona?	El sistema debería comparar el peso económico de turismo y hostelería con otros sectores, explicando límites del dato cuando el turismo esté repartido entre varias actividades.
20	¿Qué ciudades podrían estar más expuestas a una caída del turismo?	El sistema debería identificar territorios con alta dependencia turística y explicar que una mayor concentración sectorial implica más vulnerabilidad ante cambios en la demanda turística.

2.3.5. Bloque E: Energía, Transporte y Vida Diaria

Nº	Pregunta	Respuesta esperada
21	¿La electricidad afecta igual al coste de vida en todas las ciudades?	El sistema debería explicar que el precio eléctrico es común en gran parte del mercado, pero su impacto puede variar según renta, clima, consumo doméstico o estructura de hogares.
22	¿Dónde puede pesar más el gasto energético, en ciudades frías como Burgos o León o en ciudades cálidas como Sevilla o Córdoba?	El sistema debería combinar precio de energía con variables climáticas o de consumo, si están disponibles, para estimar presión energética relativa.
23	¿Qué ciudades tienen mejor acceso al transporte público para vivir sin coche?	El sistema debería comparar indicadores de transporte público, movilidad o cobertura urbana cuando existan datos disponibles y homogéneos.
24	¿Dónde puede ser más caro desplazarse cada día para trabajar?	El sistema debería combinar distancia, transporte, movilidad y coste si los datos están disponibles, indicando limitaciones cuando no haya información homogénea.
25	¿Qué ciudades tienen mejores condiciones urbanas para vivir, más allá del empleo?	El sistema debería integrar variables como zonas verdes, transporte, vivienda, seguridad, aire o servicios, explicando que la calidad urbana no depende solo de la economía.

2.3.6. Bloque F: Comparación General y Decisión Ciudadana

Nº	Pregunta	Respuesta esperada
26	Si quiero mudarme, ¿qué ciudad española parece ofrecer mejor equilibrio entre empleo y coste de vida?	El sistema debería comparar empleo, salarios o renta, vivienda y coste de vida para ofrecer una lectura equilibrada entre oportunidades y gastos.
27	¿Qué ciudad está mejorando más económicamente en los últimos años?	El sistema debería analizar la evolución de varios indicadores, como empleo, actividad económica, población, vivienda y turismo, para detectar mejora sostenida.
28	¿Qué ciudades parecen más difíciles para empezar una vida independiente?	El sistema debería observar coste de vivienda, empleo, salarios y presión urbana para identificar territorios donde la emancipación puede ser más complicada.
29	¿Qué ciudad se parece más a otra en su situación económica y urbana?	El sistema debería comparar perfiles de indicadores entre ciudades, no solo rankings, para detectar similitudes en estructura económica, coste de vida o presión turística.
30	¿Dónde viviría mejor una persona según sus prioridades: empleo, vivienda, tranquilidad o servicios?	El sistema debería permitir ponderar prioridades del usuario y explicar qué ciudades encajan mejor según los indicadores disponibles, aclarando que no se trata de una recomendación absoluta sino de una ayuda basada en datos.

Capítulo 3

Arquitectura de Datos Propuesta

La arquitectura de datos de ISEU+ se plantea como una solución escalable orientada a AWS, capaz de integrar fuentes públicas heterogéneas, transformar los datos en indicadores comparables y ofrecer resultados consultables por usuarios finales. Su diseño responde al alcance nacional del proyecto: las fuentes pueden incorporar datos de ciudades, municipios, provincias, comunidades autónomas o series estatales, siempre que exista trazabilidad entre el origen y el indicador final.

El objetivo de esta arquitectura no es únicamente almacenar información, sino construir un flujo completo de datos que permita pasar de registros públicos dispersos a indicadores analíticos útiles. Para ello, se separan las responsabilidades de ingesta, validación, transformación, agregación, almacenamiento analítico y visualización. Esta separación facilita el mantenimiento del sistema, permite auditar cada etapa y reduce el riesgo de que errores en una fuente concreta afecten a todo el análisis.

3.1. Visión general

El flujo se organiza en capas de procesamiento que separan la ingesta, la curación, el cálculo analítico y la consulta. Esta separación permite conservar los datos originales, aplicar reglas de limpieza de forma controlada y generar métricas finales sin perder la posibilidad de auditar el proceso. La arquitectura también contempla zonas de error para aislar registros defectuosos y facilitar reprocesos posteriores.

La lógica general sigue un patrón medallion, compuesto por una capa Bronze para datos brutos, una capa Silver para datos curados y una capa Gold para métricas analíticas. Este patrón resulta adecuado para ISEU+ porque las fuentes públicas suelen tener distintos formatos, frecuencias de actualización y niveles territoriales. Por ejemplo, algunas variables pueden llegar a nivel municipal, otras a nivel provincial y otras solo a nivel autonómico o estatal. La arquitectura permite almacenar esa granularidad original y documentar posteriormente qué transformaciones son necesarias para hacer comparaciones responsables.

A nivel funcional, el sistema parte de fuentes oficiales y archivos complementarios, programa procesos periódicos de ingesta, valida los datos recibidos y los transforma en datasets comparables. Después calcula indicadores y scores que se almacenan en un modelo ana-

lítico preparado para consultas. Finalmente, una capa de interpretación de preguntas y visualización permite que el usuario consulte resultados en forma de tablas, gráficos o reportes.

3.2. Arquitectura AWS actualizada

La nueva arquitectura de ISEU+ se estructura como una plataforma cloud de datos urbanos basada en servicios gestionados de AWS. El punto de entrada son las fuentes externas: APIs públicas, portales Open Data municipales, archivos CSV o JSON oficiales, cargas manuales y, solo cuando sea viable, procesos de web scraping. Estas fuentes se ejecutan de forma periódica mediante Amazon EventBridge, con una planificación mensual pensada para actualizar indicadores urbanos sin depender de una ejecución manual.

La ingesta se resuelve mediante funciones AWS Lambda que actúan como recolectores. Cada Lambda puede representar una familia de fuentes: conectores nacionales, conectores regionales o conectores municipales. Esta división es coherente con la estrategia definida para el proyecto: primero se consultan fuentes nacionales, después fuentes regionales y finalmente portales municipales como Open Data BCN, Madrid Datos Abiertos, Valencia, Málaga, Zaragoza, Bilbao o Sevilla cuando una variable no existe con suficiente granularidad en fuentes estatales.

Los datos descargados se almacenan inicialmente en Amazon S3 dentro de la zona Bronze o Raw Data Zone. Esta capa conserva los archivos originales, los JSON recibidos, los CSV descargados y los logs de ejecución. Su función no es ofrecer datos listos para análisis, sino mantener una copia trazable y reproducible de lo recibido desde cada fuente. Si una API cambia, si una descarga falla o si se detecta un error de transformación, esta capa permite volver al dato original.

Sobre la capa Bronze actúa AWS Glue Crawler, encargado de detectar esquemas, registrar datasets en el catálogo y alertar de posibles cambios de estructura. A continuación, AWS Glue ejecuta procesos de limpieza, validación y normalización que generan la capa Silver. En esta fase se validan distritos y ciudades, se normalizan variables, se estandarizan formatos, se unifican catálogos y se corrigen diferencias de unidades, fechas o nombres de campo.

La capa Gold contiene los resultados analíticos: scores por variable, scores por pilar, score global ISEU+, métricas finales y agregaciones por distrito, ciudad o región. Estos resultados se almacenan en S3 Gold y posteriormente se cargan en Amazon Redshift, que funciona como data warehouse SQL. Redshift permite organizar el modelo dimensional, optimizar consultas, crear vistas materializadas y responder preguntas analíticas de forma rápida.

Finalmente, la aplicación web y el chat se conectan a una capa de interpretación de preguntas. El modelo interpretador transforma consultas en lenguaje natural en consultas SQL, que pasan por un validador y optimizador antes de ejecutarse sobre Redshift. Esta capa protege el sistema frente a consultas costosas, errores de agregación o preguntas que intenten utilizar datos incompletos.

3.3. Justificación del almacenamiento SQL para consultas conversacionales

La decisión de cargar la capa Gold en una base SQL responde a una necesidad central del proyecto: permitir que una pregunta escrita en lenguaje natural pueda transformarse en una consulta estructurada, trazable y reproducible. Aunque los datos originales se encuentren en CSV, JSON, GeoJSON, XLS o catálogos abiertos, esos formatos no son adecuados para que una interfaz conversacional responda de manera consistente. El chat no debería tener que leer cientos de archivos raw, interpretar columnas distintas en cada fuente y decidir en tiempo real cómo combinar indicadores. Esa lógica debe resolverse antes, durante el pipeline de datos.

SQL actúa como una capa intermedia entre el modelo de lenguaje y el data lake. El modelo no consulta directamente la capa Bronze, sino una estructura analítica ya normalizada. Esto reduce la ambigüedad, mejora el rendimiento y permite controlar mejor las respuestas. Por ejemplo, si el usuario pregunta por “paro”, el sistema puede mapear esa intención a la variable `unemployed_registered`; si pregunta por “contratos”, puede consultar `contracts_registered`; y si pregunta por población, puede utilizar `population_total`. El modelo conversacional no inventa la estructura del dato, sino que utiliza un catálogo de variables ya definido.

Otra ventaja de SQL es la auditabilidad. Cada respuesta generada por la web puede asociarse a una consulta concreta, a una tabla, a una fecha de carga y a una fuente. Esto es importante porque ISEU+ no busca únicamente generar texto, sino producir respuestas defendibles basadas en datos oficiales. Si una respuesta indica que una ciudad tiene más desempleo que otra, el sistema debe poder explicar qué variable utilizó, de qué fuente procede, en qué periodo se midió y qué filtros se aplicaron.

Desde el punto de vista técnico, SQL también permite aplicar restricciones antes de ejecutar una consulta. En una arquitectura AWS, la consulta generada por el modelo pasaría por una capa de validación que comprobaría que solo se consultan tablas permitidas, que las columnas existen, que la consulta no modifica datos y que el coste de ejecución es razonable. Esta validación es más difícil si el modelo opera directamente sobre archivos sueltos.

Por tanto, la base SQL no sustituye al data lake. El data lake conserva el volumen, la trazabilidad y los archivos originales. La base SQL representa la capa optimizada para consumo analítico y conversacional. En local, esta función se valida con SQLite. En la arquitectura final, el equivalente natural sería Amazon Redshift o Athena sobre tablas catalogadas, dependiendo del volumen final y del patrón de consultas.

3.4. Flujo de consulta desde la página web

La aplicación web prevista para ISEU+ funcionaría como una capa de acceso ciudadano al modelo de datos. El usuario no necesitaría conocer nombres de columnas, fuentes oficiales o sintaxis SQL. En lugar de eso, escribiría una pregunta en lenguaje natural, por ejemplo: “¿Cómo ha evolucionado el paro en Valencia desde 2015?” o “¿Qué ciudad tiene más

contratos registrados?”.

El flujo lógico de una consulta se compone de varias etapas. Primero, la interfaz web captura la pregunta y la envía al componente conversacional. Después, el modelo interpreta la intención de la pregunta: identifica si se trata de empleo, población, renta, movilidad u otra categoría. A continuación, detecta las entidades relevantes, como ciudad, periodo temporal, variable solicitada y tipo de comparación. Con esta información, el sistema busca en el catálogo semántico qué variable del modelo corresponde a la pregunta.

Una vez identificada la variable, el modelo genera una sentencia SQL contra la tabla analítica. Esa consulta no se ejecuta directamente sin control. Antes debe pasar por una capa de validación que comprueba que sea una consulta de solo lectura, que utilice tablas permitidas y que no incluya operaciones peligrosas o innecesariamente costosas. Si la consulta es válida, se ejecuta sobre el almacén SQL y devuelve un resultado estructurado.

Finalmente, el modelo transforma el resultado SQL en una respuesta comprensible para el usuario. Esta respuesta puede incluir una frase explicativa, una tabla resumida o un gráfico en la página web. Lo importante es que el texto final no se basa únicamente en conocimiento general del modelo, sino en una consulta concreta sobre la base de datos ISEU+.

El flujo puede resumirse de la siguiente forma:

Pregunta del usuario

- > interpretación semántica
- > mapeo a variable ISEU+
- > generación SQL
- > validación de consulta
- > ejecución en SQLite/Redshift
- > respuesta textual, tabla o gráfico

Este diseño permite separar el razonamiento lingüístico del almacenamiento de datos. El modelo de lenguaje aporta flexibilidad para entender preguntas humanas, mientras que SQL aporta precisión, estructura y trazabilidad. Esta separación es especialmente importante en un proyecto de datos públicos, donde las respuestas deben ser explicables y no depender únicamente de generación textual.

3.5. Decisiones de diseño

La arquitectura se ha diseñado priorizando cuatro criterios: trazabilidad, escalabilidad, separación de responsabilidades y capacidad de reproceso. La trazabilidad es necesaria porque cada indicador debe poder relacionarse con su fuente original, fecha de descarga y proceso de transformación. La escalabilidad permite incorporar nuevas ciudades, fuentes o variables sin rediseñar todo el sistema. La separación de responsabilidades evita mezclar datos brutos, datos limpios y métricas finales en un mismo espacio. Por último, la capacidad de reproceso permite corregir errores o actualizar reglas de negocio sin perder el histórico.

Otra decisión relevante es mantener una distinción clara entre procesamiento operativo y consumo analítico. Amazon S3 actúa como almacenamiento flexible para las capas de

datos, mientras que Redshift se reserva para consultas analíticas estructuradas. De este modo, el sistema puede conservar archivos originales y, al mismo tiempo, ofrecer una base optimizada para comparaciones entre territorios.

3.6. Capas de procesamiento

- **Fuentes y ejecución:** el sistema incorpora APIs públicas, archivos manuales y procesos de extracción web. La ejecución periódica se plantea mediante Amazon EventBridge y funciones AWS Lambda para automatizar la ingesta. Esta capa debe registrar cuándo se ejecuta cada proceso, qué fuente se consulta y si la descarga finaliza correctamente.
- **Bronze Raw Layer:** Amazon S3 almacena los datos brutos, los logs de ingesta y los archivos originales en JSON, CSV u otros formatos de origen. Esta capa conserva la evidencia inicial del dato recibido y permite reconstruir el procesamiento si una regla de limpieza cambia en el futuro.
- **Catálogo y detección de esquema:** AWS Glue Crawler identifica la estructura de los datasets, detecta cambios de esquema y facilita que los datos puedan ser consultados o transformados de forma más sistemática. Este punto es importante porque las fuentes públicas pueden modificar columnas, nombres o formatos sin previo aviso.
- **Silver Curated Layer:** AWS Glue limpia, normaliza, valida y estandariza los datos. En esta fase se tratan nulos, formatos, catálogos, fechas, unidades, duplicados y posibles incompatibilidades entre fuentes. El objetivo es obtener variables homogéneas y preparadas para comparación territorial.
- **Gold Analytics Layer:** se calculan scores por variable, pilar y puntuación global ISEU+. La agregación puede realizarse por distrito, ciudad, provincia, comunidad autónoma o región, dependiendo de la granularidad disponible. Esta capa contiene los resultados listos para análisis y visualización.
- **Almacenamiento analítico:** Amazon Redshift actúa como data warehouse SQL para consultas rápidas, vistas materializadas y análisis dimensional. Aquí se organizarían tablas de hechos, dimensiones territoriales, dimensiones temporales y métricas finales.
- **Capa de consulta y visualización:** el modelo interpretador transforma preguntas de usuario en consultas, con validación y optimización antes de ejecutar SQL. La salida se presenta mediante resultados tabulares, gráficos, reportes o una aplicación web de consulta.

3.7. Estrategia jerárquica de fuentes

La arquitectura se ha ajustado para evitar que el sistema quede sesgado hacia una sola ciudad. La estrategia de recolección se organiza en tres niveles. En primer lugar se

priorizan fuentes nacionales como INE, SEPE, MITMA/MIVAU, AEMET, MITECO o datos.gob.es, ya que permiten cubrir muchas ciudades con una misma metodología. En segundo lugar se consideran fuentes regionales cuando aportan mayor detalle para una comunidad autónoma concreta. En tercer lugar se incorporan fuentes municipales, especialmente cuando la variable depende de información urbana local, como movilidad, ruido, equipamientos, zonas verdes o tráfico.

Dentro de esta estrategia, los portales municipales no sustituyen a las fuentes nacionales, sino que las complementan. Por ejemplo, una variable de empleo debe obtenerse preferentemente desde SEPE o INE si existe cobertura municipal o provincial comparable. En cambio, variables como estaciones de calidad del aire, equipamientos públicos, bicicletas compartidas o incidencias de tráfico pueden depender de portales municipales. Esta separación permite que ISEU+ sea multicidad sin convertir Barcelona en el caso dominante del modelo.

3.8. Relación con el entorno local

La implementación local desarrollada en el proyecto funciona como una validación previa de esta arquitectura cloud. Las carpetas de conectores API equivalen a la capa de ingesta, los datasets descargados representan la zona Bronze, los procesos de limpieza se corresponden con la capa Silver y la base SQLite utilizada en local anticipa el papel que tendría un almacén analítico como Redshift. Esta equivalencia permite avanzar de forma incremental: primero se comprueba que los datos pueden obtenerse, limpiarse y consultarse en local; después, esos mismos procesos pueden migrarse a servicios gestionados de AWS.

Esta aproximación reduce el riesgo del proyecto, porque no obliga a desplegar toda la infraestructura cloud desde el inicio. En su lugar, permite validar fuentes, estructura de carpetas, reglas de limpieza, consultas y preguntas ciudadanas con un coste menor. Una vez estabilizados los procesos, la migración a AWS consistiría en sustituir componentes locales por servicios equivalentes, manteniendo la misma lógica funcional.

3.9. Gestión de errores y reproceso

El diseño contempla fallos frecuentes en pipelines de datos públicos: caídas de APIs, datos incompletos, archivos corruptos, duplicados, cambios de esquema, valores inválidos, errores de unión, métricas inconsistentes o consultas costosas. Para evitar que estos problemas contaminen las capas curadas y analíticas, la arquitectura incorpora zonas de error en S3 y un flujo de corrección/reproceso de datos.

Cuando una ingesta falla o un registro no supera las validaciones, el dato no debería eliminarse directamente. En su lugar, se enviaría a una zona de error con información suficiente para diagnosticar el problema: fuente, fecha, tipo de error, regla incumplida y archivo afectado. Esta estrategia permite revisar errores sin perder información y facilita ejecutar reprocesos cuando se corrige la causa del fallo.

3.10. Gobernanza y calidad del dato

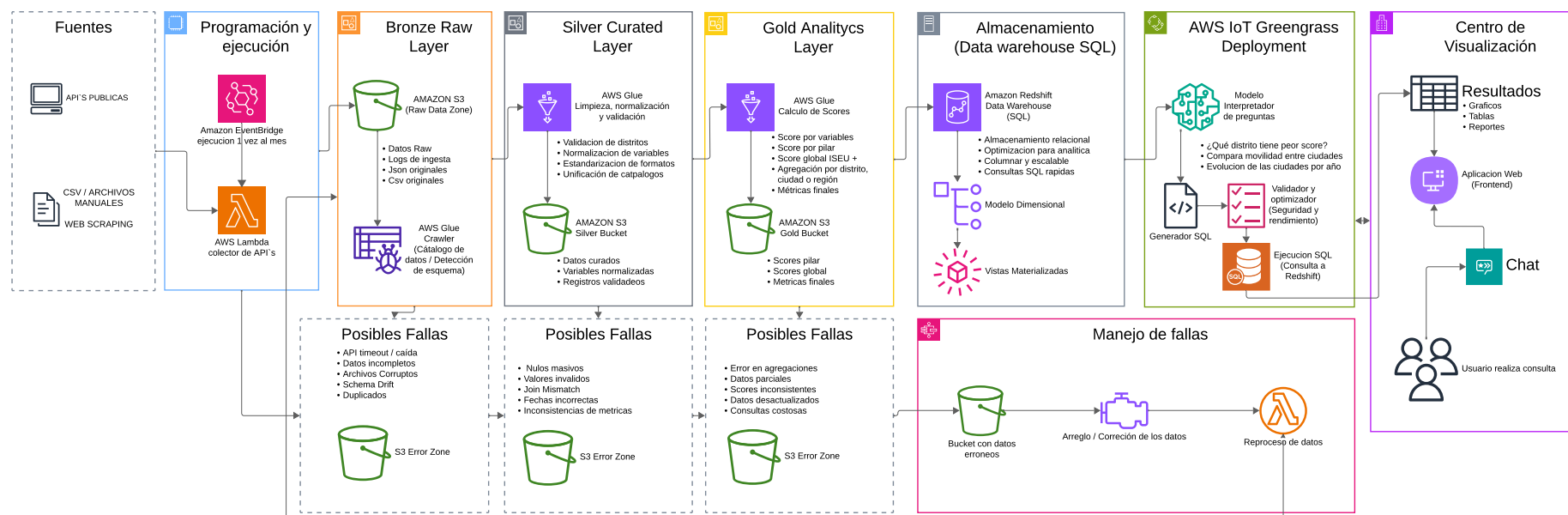
Para que ISEU+ pueda apoyar comparaciones entre ciudades españolas, la calidad del dato debe tratarse como una parte central de la arquitectura. Cada variable debería contar con metadatos mínimos: fuente, fecha de actualización, nivel territorial, unidad de medida, frecuencia, método de cálculo y limitaciones conocidas. Esta información es especialmente importante cuando se comparan indicadores que no siempre existen con la misma granularidad en todas las localidades.

La arquitectura también debe permitir distinguir entre ausencia de dato, dato no aplicable y dato pendiente de actualización. Esta distinción evita conclusiones incorrectas y ayuda a que el sistema comunique sus límites de forma transparente. En un contexto de análisis económico urbano, tan importante como calcular un score es explicar con qué datos se ha calculado y qué restricciones presenta.

Figura 3.1

Arquitectura AWS propuesta para la plataforma ISEU+

ARQUITECTURA AWS - PLATAFORMA ISEU+



Nota. El diagrama representa la arquitectura AWS actualizada: fuentes públicas, ejecución mensual con EventBridge, recolección mediante Lambda, almacenamiento S3 Bronze/Silver/Gold, transformaciones con Glue, catálogo de datos, zonas de error, Redshift, modelo dimensional y capa de consulta web.

Capítulo 4

Modelo y Flujo de Datos

Además de la arquitectura cloud, ISEU+ necesita un modelo lógico que explique cómo se organiza la información desde que se identifica una fuente hasta que se calculan los resultados finales. El diagrama de flujo de datos define esa estructura interna y permite conectar la capa técnica de AWS con las tablas, entidades y procesos que sostienen el análisis económico urbano.

Este capítulo describe el recorrido del dato dentro del sistema. La idea principal es que cada valor utilizado por ISEU+ debe poder seguirse desde su fuente original, pasando por los registros de ingesta y las tablas de datos brutos, hasta llegar a las variables curadas y los scores finales. Esta trazabilidad es importante porque el proyecto compara territorios y, por tanto, necesita justificar qué fuente alimenta cada indicador, cómo se ha transformado y qué versión del cálculo se ha utilizado.

4.1. Principios del modelo de datos

El modelo se organiza alrededor de tres principios: separación entre catálogos y datos observados, conservación del histórico y control explícito de errores. Los catálogos describen elementos relativamente estables, como fuentes, variables, pilares, ciudades y distritos. Los datos observados representan valores concretos descargados o calculados en una fecha determinada. El histórico permite conservar versiones de scores y procesos, mientras que las tablas de errores documentan incidencias detectadas durante la ingesta o transformación.

Esta separación evita que el sistema dependa de una única tabla monolítica. En lugar de mezclar fuentes, variables, territorios, valores y resultados, cada entidad cumple una función concreta. Esto facilita ampliar el sistema cuando se incorporen nuevas fuentes de datos, nuevas ciudades o nuevos indicadores económicos.

4.2. Modelo dimensional actualizado

El nuevo modelo de datos se define como un esquema multicidadad orientado a análisis y consulta. Su propósito es conectar tres necesidades del proyecto: conservar la trazabilidad

del dato desde la fuente original, normalizar observaciones heterogéneas para que sean comparables y calcular resultados finales que puedan consultarse desde SQL o desde una interfaz conversacional.

El modelo se organiza en tablas maestras, tablas de mapeo, tablas operativas de ingesta, tablas de datos y tablas analíticas. Las tablas maestras describen elementos relativamente estables, como fuentes, variables, pilares, ciudades y distritos. Las tablas de mapeo explican qué fuente alimenta cada variable y qué prioridad tiene cada origen. Las tablas operativas registran ejecuciones, metadatos y errores. Las tablas de datos separan la información raw de la información curada. Finalmente, las tablas analíticas almacenan scores por pilar, score global e histórico de cálculo.

Tabla / Entidad	Función dentro del modelo
FUENTES	Catálogo de fuentes de datos. Registra nombre, tipo de acceso, descripción, endpoint, institución responsable, nivel territorial, cobertura geográfica, frecuencia, formato y fecha de registro. Permite distinguir fuentes nacionales, regionales y municipales.
METADATOS_ FUENTE	Describe la estructura esperada de cada fuente: campos, tipos de dato, obligatoriedad y fecha del último cambio. Sirve para detectar cambios de esquema antes de cargar datos incorrectos.
PILARES	Define las dimensiones principales del índice ISEU+, su descripción y orden de presentación. Cada variable se asocia a un pilar para permitir el cálculo agregado.
VARIABLES	Catálogo maestro de indicadores. Incluye descripción, unidad, tipo, sentido positivo o negativo, fuente principal y método de cálculo. Es la tabla que permite mantener una definición estable de cada variable aunque cambie el nombre del campo en una fuente externa.
VARIABLE_FUENTE	Tabla puente entre variables y fuentes. Permite que una variable tenga varias fuentes posibles y que una fuente alimente distintas variables. Conserva el nombre de variable en la fuente, el campo original, el método de extracción y la frecuencia de actualización.
SOURCE_PRIORITY	Define la prioridad de uso por variable y ciudad. Permite seleccionar primero una fuente nacional, luego una regional y finalmente una municipal, registrando la razón de la decisión y un score de calidad.
CIUDADES	Catálogo de ciudades analizadas. Incluye país, región, población, zona horaria y fecha de creación. Funciona como dimensión territorial principal para comparación multicidad.
DISTRITOS	Catálogo de distritos o zonas internas de cada ciudad. Permite almacenar indicadores con granularidad urbana cuando el portal municipal lo permite.

Tabla / Entidad	Función dentro del modelo
LOG_INGESTA	Historial de cada ejecución de ingesta. Registra fuente, fecha, estado, registros leídos, aceptados y rechazados, archivo de origen y mensajes de error.
DATOS_RAW	Representa la capa Bronze. Guarda referencias al archivo original, formato recibido, número de registros, fecha de carga y hash del archivo. Su objetivo es conservar el dato bruto sin alterar.
ERRORES_DATOS	Tabla de auditoría para registros o procesos con error. Permite clasificar el tipo de error, describirlo, conservar el registro afectado y documentar la acción tomada.
DATOS_CURADOS	Representa la capa Silver. Contiene valores originales y normalizados, variable asociada, distrito, fuente, método de cálculo y calidad del dato. Es la base para comparaciones homogéneas.
SCORES_PILAR	Representa resultados Gold por pilar. Calcula un score entre 0 y 100 por ciudad, distrito, pilar y fecha, indicando cuántas variables válidas participaron.
SCORES_ISEU	Almacena el score final ISEU+ por ciudad o distrito. Incluye score global, clasificación, fecha y versión metodológica.
FACT_SCORES_HISTORICO	Conserva el histórico de cálculos, versiones, usuario o proceso responsable y notas. Permite auditar cambios metodológicos y reproducir resultados anteriores.

Esta estructura es más robusta que una tabla única de indicadores porque separa la definición conceptual de la variable, la fuente que la alimenta, el dato raw recibido, el dato curado y el resultado analítico final. Además, introduce una pieza clave para el alcance multicuidad: la tabla `SOURCE_PRIORITY`. Gracias a ella, el sistema puede decidir de forma explícita qué fuente usar para cada ciudad y variable, evitando mezclar datos nacionales, regionales y municipales sin control metodológico.

4.3. Catálogos maestros

Las tablas maestras representan el vocabulario básico del sistema. La tabla de fuentes registra el origen de los datos, su tipo, institución responsable, URL o endpoint, formato y frecuencia de actualización. Esta tabla permite diferenciar entre APIs públicas, archivos CSV, scraping web o bases de datos externas.

La tabla de variables define qué mide cada indicador: nombre, descripción, unidad, tipo, sentido del indicador y pilar principal. El campo de sentido resulta relevante porque no todas las variables se interpretan igual. En algunas métricas, un valor alto puede ser positivo, como empleo o renta; en otras, puede ser negativo, como paro, esfuerzo de alquiler o presión turística.

La relación entre variables y fuentes se gestiona mediante una tabla intermedia. Esta tabla

permite mapear una misma variable conceptual a diferentes nombres de campo según la fuente. Por ejemplo, una fuente puede llamar a una variable `paro_registrado`, otra `desempleo` y otra presentar el dato dentro de una estructura más compleja. Este mapeo evita que los procesos de limpieza dependan de nombres rígidos y facilita incorporar nuevas fuentes sin alterar el modelo completo.

4.4. Territorios y niveles de análisis

El modelo distingue entre ciudades y distritos. La tabla de ciudades funciona como catálogo territorial principal e incluye información como nombre, país, región, población y zona horaria. La tabla de distritos se vincula a ciudades mediante una clave foránea e incorpora campos como nombre, código, tipo, población y superficie.

Aunque el diagrama utiliza el concepto de distrito porque muchas métricas urbanas pueden analizarse a ese nivel, el diseño puede extenderse a otros niveles territoriales. Para el alcance nacional de ISEU+, esta flexibilidad es importante: algunas ciudades tienen distritos oficiales comparables, mientras que otras pueden requerir trabajar a nivel municipal, provincial o autonómico según la disponibilidad de datos.

4.5. Ingesta y datos brutos

La tabla de log de ingesta registra cada ejecución del sistema. Incluye la fuente consultada, fecha de ingesta, estado del proceso, registros leídos, registros aceptados, registros rechazados, archivo de origen y mensaje de error. Esta información permite saber si una descarga funcionó, cuántos datos se incorporaron y qué problemas aparecieron.

Los datos brutos se almacenan en la tabla o zona de datos raw. Esta capa mantiene la referencia a la fuente y al log de ingesta, la ruta del archivo original, el formato recibido, la cantidad de registros, la fecha de carga y un hash del archivo. El hash ayuda a detectar cambios, duplicados o reprocesamientos sobre un mismo archivo.

Mantener los datos raw sin transformar permite auditar el proceso y reconstruir resultados. Si una regla de normalización cambia, el sistema puede volver al archivo original y reprocesarlo sin depender de una descarga nueva.

4.6. Metadatos y validación

La tabla de metadatos de fuente describe la estructura esperada de cada origen: nombre de campo, tipo de dato, descripción, obligatoriedad y fecha del último cambio. Esta tabla es clave para detectar cambios de esquema. Si una API cambia el nombre de una columna o modifica el tipo de un campo, el sistema puede identificar la desviación antes de contaminar los datos curados.

Los errores detectados durante la ingesta o transformación se registran en una tabla específica de errores. Cada error conserva la referencia al log de ingesta, el tipo de error,

la descripción, el registro afectado, la acción tomada y la fecha. Este diseño permite revisar incidencias sin perder información y diferencia entre errores corregibles, errores que requieren intervención manual y registros que deben quedar excluidos del cálculo.

4.7. Datos curados y normalización

Los datos curados representan la capa Silver del modelo. Aquí cada observación se vincula con una variable, un territorio, una fecha y una fuente. Además del valor original, se conserva un valor normalizado, el método de cálculo y una puntuación de calidad del dato. Esta estructura permite comparar variables aunque provengan de fuentes con formatos o unidades diferentes.

La normalización no consiste únicamente en cambiar formatos. También implica validar unidades, corregir fechas, unificar catálogos, resolver duplicados y decidir cómo tratar valores faltantes. En un sistema de análisis económico urbano, estas decisiones afectan directamente a la interpretación de resultados. Por eso el modelo conserva tanto el valor original como el valor normalizado.

4.8. Cálculo de scores

Los scores se organizan en dos niveles. Primero se calculan scores por pilar, vinculados a ciudad, distrito, pilar y fecha. Después se calcula un score global ISEU+, que resume el estado económico urbano de un territorio en una escala interpretable. El modelo incluye campos de versión del método para que los resultados puedan compararse incluso si la fórmula evoluciona durante el desarrollo del proyecto.

La clasificación final permite traducir los scores numéricos a categorías más comprensibles, como A, B, C, D o E. Esta categorización puede ser útil para visualizaciones y reportes, siempre que se explique cómo se han definido los rangos y qué limitaciones tiene la comparación.

4.9. Histórico, auditoría y reproducibilidad

La tabla de histórico de scores conserva las versiones calculadas, fecha de cálculo, usuario o proceso responsable y notas asociadas. Esta capa es importante porque los indicadores públicos se actualizan con el tiempo y los resultados de ISEU+ pueden cambiar cuando se incorporan datos nuevos o se corrigen datos anteriores.

Conservar el histórico evita sobrescribir resultados sin control. También permite responder preguntas como qué score tenía una ciudad en una fecha determinada, cuándo se recalculó un indicador o qué versión metodológica produjo un resultado concreto. Esta trazabilidad aporta rigor al proyecto y lo acerca a una arquitectura analítica profesional.

4.10. Relación entre flujo lógico y arquitectura AWS

El modelo lógico se corresponde con la arquitectura AWS descrita en el capítulo anterior. Las tablas de fuentes, variables, metadatos y territorios actúan como catálogos maestros. Los logs de ingesta y datos raw se alinean con la capa Bronze. Los datos curados y normalizados se corresponden con la capa Silver. Los scores por pilar, el score global y el histórico forman parte de la capa Gold y del almacenamiento analítico.

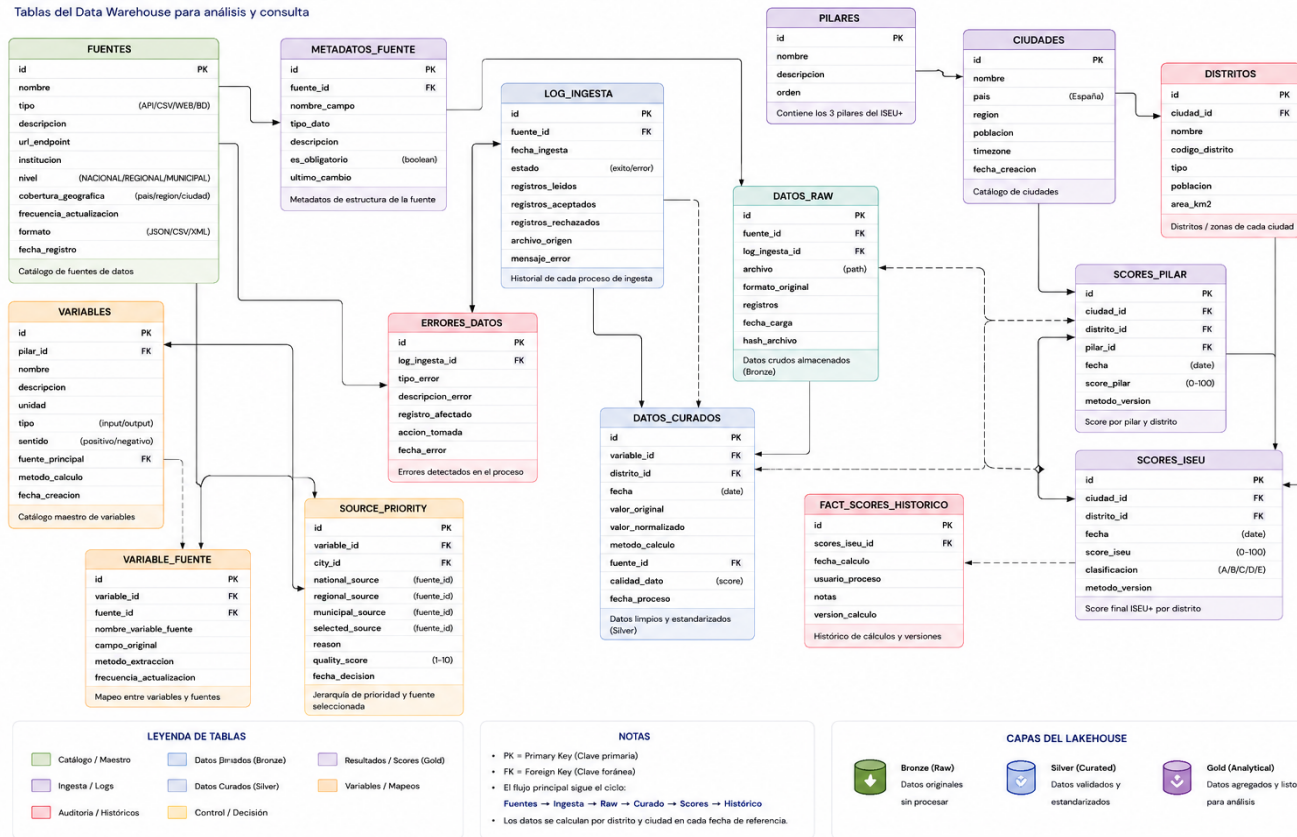
Esta correspondencia permite que el proyecto local tenga una ruta clara de evolución hacia la nube. En local, las tablas pueden materializarse en SQLite o archivos procesados. En AWS, esas mismas entidades pueden almacenarse en S3, Glue Catalog y Redshift, manteniendo la lógica conceptual del sistema.

Figura 4.1

Modelo lógico y flujo de datos de ISEU+

MODELO DE DATOS – ISEU+ (MULTICIUDAD – ESPAÑA)

Tablas del Data Warehouse para análisis y consulta



Nota. El diagrama muestra el modelo de datos multiciudad de ISEU+: fuentes, metadatos de fuente, variables, pilares, ciudades, distritos, mapeos variable-fuente, prioridad de fuentes, logs de ingesta, datos raw, datos curados, errores, scores por pilar, score global e histórico de cálculo.

Capítulo 5

Capítulo 2: Creación del Entorno Local de Desarrollo y Validación

En este capítulo se describe la creación del entorno local utilizado para desarrollar y validar las primeras fases de ISEU+. Antes de llevar el proyecto a una arquitectura AWS, se ha construido una versión local que permite probar la ingesta de datos, la estructura de carpetas, los conectores API, el almacenamiento temporal, la limpieza de información y las primeras pruebas de interacción con el sistema.

El objetivo de esta fase no es construir la versión final de la plataforma, sino comprobar que el flujo de datos funciona correctamente desde el origen hasta su preparación para el análisis. Para ello, se han organizado los componentes principales del proyecto en módulos diferenciados: conectores API, datos descargados, procesos de limpieza, datasets estructurados, página web de prueba y entorno conversacional local.

Capítulo 6

Estructura General del Proyecto

La estructura actual del proyecto se ha reorganizado alrededor del `intento 3`, que funciona como versión más ordenada de la capa de recolección. Esta versión separa los conectores generales de los conectores municipales y utiliza una estructura tipo lakehouse local, donde los datos descargados se almacenan inicialmente en una capa Bronze. Esta decisión permite mantener los archivos originales sin transformar y facilita que, en una fase posterior, puedan pasar por procesos Silver y Gold.

Carpeta / Archivo	Función dentro del proyecto
<code>api_clients/intento 3/APIS/</code>	Contiene los conectores principales del intento 3. Incluye scripts para fuentes generales como INE, MITMA, SEPE, AEMET y catálogos, además de un orquestador <code>run_all.py</code> .
<code>api_clients/intento 3/APIS/municipios/</code>	Contiene los conectores municipales específicos. Cada archivo representa un portal Open Data de una ciudad: Barcelona, Madrid, Valencia, Málaga, Zaragoza, Bilbao y Sevilla.
<code>api_clients/intento 3/data_lake/bronze/</code>	Capa Bronze local. Guarda los datos raw tal como son descargados desde cada fuente, sin limpieza ni transformación.
<code>api_clients/intento 3/data_lake/bronze/apis/</code>	Almacena los datos crudos de las APIs nacionales o generales. Cada fuente tiene su propia subcarpeta: <code>ine</code> , <code>mitma</code> , <code>sepe</code> , <code>catalogos</code> y <code>aemet</code> .
<code>api_clients/intento 3/data_lake/bronze/municipios/</code>	Almacena los catálogos y resultados de búsqueda de los portales municipales. Cada ciudad tiene su propia subcarpeta.
<code>api_clients/intento 3/reports/</code>	Guarda reportes de ejecución, como <code>apis_run.json</code> y <code>municipios_run.json</code> , donde se registra qué conectores se ejecutaron, qué salidas generaron y cuál fue su resultado.
<code>pag_web/</code>	Contiene la página web de prueba utilizada para validar la interfaz inicial del sistema.

Carpeta / Archivo	Función dentro del proyecto
pag_web/Assets/	Incluye archivos CSS y JavaScript de la interfaz web.
pag_web/Procesos/	Contiene scripts relacionados con limpieza, análisis y preparación de datasets.
pag_web/Procesos/Datasets/	Guarda los datos limpios, la base SQLite y los resúmenes de carga.
pag_web/LMlocal/	Contiene las pruebas de conexión con LM Studio y el servidor local de consulta.
datos de validacion/	Incluye documentos HTML usados para probar preguntas, respuestas y validación funcional.
plantillas/	Contiene las plantillas utilizadas para la elaboración de la memoria del TFM.

Esta organización permite trabajar de forma modular. Los conectores del intento 3 se encargan de obtener datos y catálogos, la capa Bronze conserva las respuestas originales, los reportes permiten auditar las ejecuciones y la página web representa la futura capa de consumo. La estructura local replica parcialmente la arquitectura AWS descrita anteriormente: las APIs equivalen a la capa de ingesta, `data_lake/bronze` equivale a S3 Bronze y los reportes locales anticipan los logs de ingesta.

La capa Bronze local se ha organizado separando las fuentes nacionales o generales de las fuentes municipales. Esta división evita mezclar datos de distinta naturaleza y facilita aplicar la estrategia jerárquica descrita en la arquitectura: primero se consultan fuentes estatales o nacionales; después, si falta granularidad o una variable urbana concreta, se recurre a portales municipales. La estructura de salida queda definida de la siguiente forma:

```
api_clients/intento 3/data_lake/bronze/
apis/
  ine/
  mitma/
  sepe/
  catalogos/
  aemet/
municipios/
  barcelona/
  madrid/
  valencia/
  sevilla/
  bilbao/
  malaga/
  zaragoza/
```

Cada carpeta de `apis/<fuente>` incluye los archivos raw descargados y un `manifest_raw.json`. Este manifiesto registra la fuente, la fecha de ingesta, las URLs consultadas, el estado de cada recurso y las rutas de salida. De esta forma, la ejecución local no se limita a descargar archivos, sino que también deja una evidencia mínima para auditoría y reproceso.

6.1. Creación de los Conectores API

Uno de los pasos principales del intento 3 ha sido organizar la recolección en conectores independientes. Cada conector tiene una responsabilidad concreta: consultar una fuente, descargar el catálogo o los datos disponibles y guardar el resultado en la capa Bronze. Esta separación facilita depurar errores, añadir nuevas ciudades y mantener el criterio jerárquico definido en la arquitectura: fuentes nacionales primero y fuentes municipales como complemento cuando se requiere granularidad urbana.

Los conectores nacionales se ejecutan mediante el orquestador `run_all.py`. Este script permite lanzar todas las fuentes generales o seleccionar solo algunas por argumento. En la ejecución sin argumentos se consultan INE, MITMA/MIVAU, SEPE, datos.gob.es y AEMET. Las salidas se guardan en `data_lake/bronze/apis/<fuente>/` y el resumen global se registra en `reports/apis_run.json`.

API / Fuente	Datos o recursos capturados	Salida Bronze y estado de ejecución
INE	Instituto Nacional de Estadística. Se descargan tablas municipales de población por sexo y población por país de nacimiento, en CSV, XLSX y metadatos recientes.	Guarda los recursos en <code>bronze/apis/ine/</code> y registra <code>manifest_raw.json</code> . Estado: OK, con 6 recursos descargados.
MITMA / MIVAU	Open Data Movilidad. Se descarga el portal de referencia, documentación técnica y la relación territorial de distritos MITMA.	Guarda <code>portal_opendata_movilidad.html</code> , documentación PDF, <code>relaciones_distrito_mitma.csv</code> y <code>manifest_raw.json</code> . Estado: OK, con 3 recursos descargados.
SEPE	Datos abiertos de paro registrado, contratos registrados y demandantes de empleo por municipio, recuperados desde distribuciones oficiales publicadas en datos.gob.es y sede SEPE.	Guarda CSV anuales desde 2010 en <code>bronze/apis/sepe/</code> . Estado: OK, con 50 recursos descargados.
Catálogos nacionales	API pública de datos.gob.es para localizar datasets relacionados con población, renta, empleo, vivienda, movilidad, calidad del aire, zonas verdes y accidentes.	Guarda búsquedas JSON por tema y <code>manifest_raw.json</code> en <code>bronze/apis/catalogos/</code> . Estado: OK, con 9 búsquedas registradas.

API / Fuente	Datos o recursos capturados	Salida Bronze y estado de ejecución
AEMET	AEMET OpenData. El conector queda preparado para descargar observación y predicción cuando exista una clave <code>AEMET_API_KEY</code> .	Guarda documentación de OpenData y <code>manifest_raw.json</code> . Estado: <code>SKIPPED_AUTH</code> , porque no se configuró clave de API.

La ejecución nacional se realiza con el siguiente comando:

```
python "api_clients\intento 3\APIS\run_all.py"
```

También pueden ejecutarse fuentes concretas, por ejemplo:

```
python "api_clients\intento 3\APIS\run_all.py" ine sepe catalogos
```

El resultado de la ejecución queda registrado en `reports/apis_run.json`. En la prueba local realizada, INE, MITMA, SEPE y catálogos finalizaron correctamente. AEMET quedó pendiente de clave de API, por lo que no descarga datos meteorológicos reales hasta configurar `AEMET_API_KEY`. Esta información se conserva en Bronze para diferenciar entre una fuente correctamente descargada y una fuente que requiere credenciales externas.

En la carpeta `APIS/municipios` se han creado conectores específicos para los portales Open Data de las ciudades piloto. El objetivo inicial de estos conectores no es todavía calcular indicadores finales, sino descubrir datasets disponibles, guardar los catálogos originales y preparar la futura selección de variables. Por tanto, esta etapa funciona como una ingesta Bronze de catálogos municipales.

Script	Fuente consultada	Salida generada
<code>api_barcelona.py</code>	Open Data BCN	Ejecuta búsquedas sobre el catálogo CKAN usando términos ISEU+ como población, renta, vivienda, movilidad, tráfico, calidad del aire, ruido, zonas verdes, accidentes y equipamientos. Guarda <code>catalog_search_raw.json</code> en la carpeta Bronze de Barcelona.
<code>api_madrid.py</code>	Madrid Datos Abiertos	Descarga el catálogo oficial DCAT/RDF de Madrid. Guarda <code>catalog_raw.rdf</code> y metadatos de catálogo en la carpeta Bronze de Madrid.

Script	Fuente consultada	Salida generada
api_valencia.py	Open Data Valencia	Consulta el catálogo CKAN de Valencia mediante búsquedas temáticas. Guarda los resultados raw en la carpeta Bronze de Valencia.
api_malaga.py	Datos Abiertos Málaga	Consulta el catálogo CKAN de Málaga mediante los mismos términos urbanos. Guarda los resultados raw en la carpeta Bronze de Málaga.
api_zaragoza.py	Datos Abiertos Zaragoza	Descarga el catálogo JSON del portal de Zaragoza. Guarda el catálogo raw en la carpeta Bronze de Zaragoza.
api_bilbao.py	Bilbao Open Data	Descarga el catálogo o página base del portal como HTML cuando no existe una API CKAN homogénea. Guarda el HTML raw y sus metadatos en Bronze.
api_sevilla.py	Centro de Datos Urbanos de Sevilla	Descarga el catálogo ArcGIS Hub como JSON desde <code>collections/all/items</code> . Guarda recursos geográficos y metadatos en Bronze para su posterior normalización.
run_municipios.py	Orquestador municipal	Permite ejecutar una, varias o todas las APIs municipales y genera el reporte <code>reports/municipios_run.json</code> .

Los resultados se guardan siguiendo una ruta común:

```
api_clients/intento 3/data_lake/bronze/municipios/<ciudad>/
```

Tras ejecutar el orquestador municipal, se generan salidas raw separadas por ciudad y un informe global de ejecución. Entre los archivos producidos se encuentran:

- data_lake/bronze/municipios/barcelona/catalog_search_raw.json
- data_lake/bronze/municipios/madrid/catalog_search_raw.json
- data_lake/bronze/municipios/valencia/catalog_search_raw.json
- data_lake/bronze/municipios/malaga/catalog_search_raw.json
- data_lake/bronze/municipios/zaragoza/catalog_raw.json

- `data_lake/bronze/municipios/bilbao/catalog_raw.html`
- `data_lake/bronze/municipios/sevilla/catalog_raw.json`
- `reports/municipios_run.json`

La ejecución municipal se lanza con:

```
python "api_clients\intento 3\APIS\municipios\run_municipios.py"
```

En la prueba local, Barcelona, Madrid, Valencia, Málaga, Zaragoza y Sevilla finalizaron correctamente a nivel de catálogo. Madrid se integró mediante la API CKAN de `datos.madrid.es`, lo que permite descubrir recursos estructurados de forma automática. Sevilla se integró mediante el catálogo ArcGIS Hub del Centro de Datos Urbanos, que publica recursos geográficos y servicios *FeatureServer*. Bilbao quedó como ejecución parcial, ya que su portal se captura como HTML y no expone, en esta fase, un catálogo estructurado estable comparable a CKAN o ArcGIS Hub. Aun así, el HTML raw y sus metadatos quedan conservados en la capa Bronze, manteniendo la trazabilidad de la fuente consultada.

Además de recolectar catálogos, se incorporó una descarga controlada de recursos municipales. En una ejecución de prueba con límite de veinte recursos por ciudad, Madrid detectó 4.778 recursos y descargó correctamente 20, Sevilla detectó 80 recursos y descargó correctamente 20, mientras que Bilbao no produjo recursos estructurados descargables desde el HTML capturado. Este resultado no invalida Bilbao como fuente futura, pero sí obliga a documentarla como limitación técnica y a reservar su integración completa para una fase con parser específico o endpoint oficial más estable.

Esta forma de almacenamiento permite auditar de dónde procede cada dato y diferenciar claramente entre datos originales y datos procesados. En una fase posterior, los archivos Bronze servirán como entrada para la capa Silver, donde se seleccionarán datasets útiles, se normalizarán campos y se mapearán variables hacia el modelo de datos multicidadad.

Esta organización permite trabajar de forma modular. Los conectores se encargan de obtener los datos, los procesos de limpieza los transforman, los datasets preparados sirven como base de análisis y la página web permite comprobar cómo podría consultar la información un usuario final.

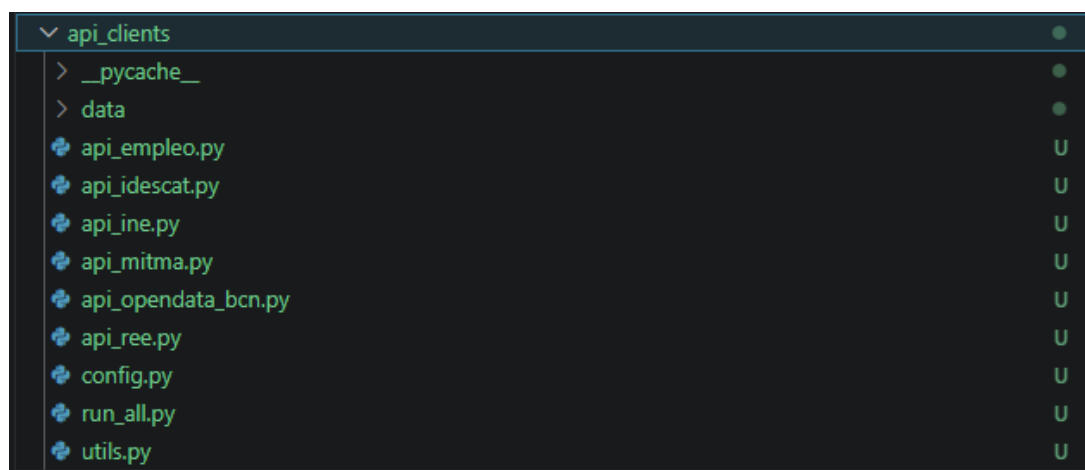
6.2. Pipeline reproducible de limpieza y carga SQL

Una vez estabilizada la recolección de datos del `intento 3`, se implementó un pipeline local reproducible para pasar desde datos brutos hasta una base SQL consultable. El objetivo de esta etapa fue ordenar el tratamiento de datos en capas, evitar que el repositorio dependa de archivos manuales y permitir que otra persona pueda ejecutar el proceso mediante scripts `.py`.

La lógica implementada sigue el patrón Bronze, Silver y Gold. La capa Bronze conserva los datos descargados sin transformación. La capa Silver limpia y normaliza las fuentes

Figura 6.1

Estructura de la carpeta de clientes API del proyecto ISEU+



Nota. Captura del entorno local de desarrollo del proyecto.

prioritarias. La capa Gold genera una tabla común de indicadores preparada para consultas SQL. Finalmente, estos indicadores se cargan en una base SQLite local, que representa el equivalente funcional de un futuro data warehouse analítico en AWS.

Esta separación es importante porque el proyecto maneja dos necesidades distintas. Por un lado, necesita conservar volumen y detalle para justificar el proceso de ingesta y permitir reprocesos. Por otro lado, necesita una capa consultable y estable para que el chat de la página web pueda responder preguntas sin ambigüedad. Si todos los archivos raw se cargaran directamente en el chat o en una única tabla sin normalización, el sistema tendría dificultades para saber qué columnas representan una variable, qué unidad utiliza cada dato o qué nivel territorial corresponde a cada registro.

Por este motivo, el pipeline no debe interpretarse como una pérdida de datos, sino como una organización por niveles de uso. Bronze funciona como repositorio de evidencia y volumen. Silver funciona como zona de limpieza y detalle operativo. Gold funciona como capa semántica y analítica para consumo. La reducción de millones de registros raw a miles de indicadores Gold es una decisión metodológica: evita que las ciudades con más portales abiertos tengan más peso simplemente porque publican más archivos.

6.2.1. Scripts implementados

El pipeline se incorporó dentro de la carpeta `api_clients/intento 3/`. La estructura nueva permite ejecutar cada fase de forma independiente o lanzar todo el flujo mediante un orquestador general.

Figura 6.2
Estructura local del pipeline reproducible ISEU+

```

ISEU+ pipeline local
> tree api_clients/intento 3 /F
api_clients/intento 3/
  APIS/
    run_all.py           # Ejecuta INE, MITMA, SEPE, catalogos y AEMET
    api_ine.py           # Poblacion municipal INE
    api_sepe.py          # Paro, contratos y demandantes por municipio
    api_mitma.py         # Open Data Movilidad y relacion territorial
    api_catalogos.py     # Busqueda oficial en datos.gob.es
    api_aemet.py         # Preparado para AEMET_API_KEY
  municipios/
    run_municipios.py    # Orquestador municipal
    download_recursos.py # Descarga recursos estructurados
    api_barcelona.py, api_valencia.py, api_zaragoza.py, ...
  pipeline/
    01_inventory_bronze.py # Perfilado de Bronze
    02_clean_silver.py    # Limpieza y normalizacion
    03_build_gold.py      # Indicadores comparables
    04_build_sqlite.py    # Carga SQL estructurada
  run_pipeline.py        # Orquestador Bronze -> Silver -> Gold -> SQL

```

Nota. Captura generada a partir de la estructura real de scripts del repositorio local.

Script	Función dentro del pipeline
run_pipeline.py	Orquesta el flujo completo. Puede ejecutar las APIs, inventariar Bronze, generar Silver, construir Gold y crear SQLite. También permite usar <code>-skip-collect</code> para reutilizar datos ya descargados.
pipeline/01_inventory_bronze.py	Recorre la capa Bronze, identifica archivos, formatos, extensiones, tamaño, columnas y filas detectadas. Genera <code>reports/inventory_bronze.json</code> y <code>reports/inventory_bronze.csv</code> .
pipeline/02_clean_silver.py	Limpia fuentes prioritarias. Normaliza columnas, codificaciones, fechas y números. Filtra INE y SEPE a las ciudades objetivo. Genera tablas Silver por fuente.
pipeline/03_build_gold.py	Convierte Silver en indicadores comparables con un esquema común. Agrega los datos municipales para evitar que una ciudad pese más solo por tener más registros brutos.

Script	Función dentro del pipeline
pipeline/04_build_sqlite.py	Carga la capa Gold en SQLite y añade tablas enriquecidas para consulta conversacional: <code>indicadores</code> , <code>semantic_observations</code> , tablas <code>silver_*</code> y <code>sql_table_catalog</code> . También copia la base resultante a <code>pag_web/Procesos/Datasets/iseu_datos.sqlite</code> , que es la ruta consumida por la web local.

El flujo completo puede ejecutarse con:

```
python "api_clients\intento 3\run_pipeline.py"
```

Cuando los datos Bronze ya existen, la etapa de recolección puede omitirse para repetir solo limpieza, Gold y SQL:

```
python "api_clients\intento 3\run_pipeline.py" --skip-collect
```

También se puede ejecutar cada fase manualmente:

```
python "api_clients\intento 3\pipeline\01_inventory_bronze.py"
python "api_clients\intento 3\pipeline\02_clean_silver.py"
python "api_clients\intento 3\pipeline\03_build_gold.py"
python "api_clients\intento 3\pipeline\04_build_sqlite.py"
```

6.2.2. Resumen cuantitativo por fase

La ejecución local del pipeline deja trazabilidad numérica en cada etapa. La capa Bronze mide el volumen recibido o inventariado; Silver mide el volumen limpio y seleccionado; Gold mide los indicadores comparables; y SQLite mide la base preparada para consulta por la página web y el modelo conversacional.

6.2.3. Resultados de recolección e inventario Bronze

La capa Bronze quedó organizada en dos grandes grupos: fuentes nacionales o generales en `bronze/apis/` y fuentes municipales en `bronze/municipios/`. La ejecución local generó un volumen importante de archivos raw y metadatos de control. Tras incorporar Madrid y Sevilla al flujo municipal, el inventario detectó 536 archivos en Bronze y 6.947.862 filas CSV identificadas.

Por fuente, Barcelona fue la ciudad con mayor cantidad de recursos descargados, seguida de Valencia y Zaragoza. Esta diferencia no se interpreta como mayor importancia analítica,

Figura 6.3

Ejecución local del pipeline Bronze, Silver, Gold y SQLite

```
●●● Ejecucion reproducible del pipeline

> python "api_clients\intento 3\run_pipeline.py" --skip-collect
Pipeline: OK

=== inventory_bronze ===
Archivos Bronze: 492
Filas CSV detectadas: 6.519.513
Tamano Bronze aprox.: 1.0 GB

=== clean_silver ===
ine/poblacion_municipal.csv: 35 filas
ine/poblacion_pais_nacimiento_total.csv: 0 filas
sepe/paro_registrado.csv: 1.281 filas
sepe/contratos_registrados.csv: 1.242 filas
sepe/demandantes_empleo.csv: 1.372 filas
mitma/relaciones_distrito_mitma.csv: 10.494 filas
municipios/municipal_prioritario.csv: 161.275 filas

=== build_gold ===
Indicadores Gold: 3.975

=== build_sqlite ===
SQLite: data_lake\gold\iseu_indicadores.sqlite
Filas cargadas: 3.975
```

Nota. La captura resume una ejecución local del flujo completo, incluyendo inventario Bronze, generación Silver, construcción Gold y carga de SQLite.

Tabla 6.5: Volumen de datos por fase del pipeline local

Fase	Salida principal	Volumen	Uso dentro del sistema
APIs nacionales	<code>bronze/apis</code>	68 recursos OK	Ingesta de INE, MITMA, SEPE y catálogos; AEMET queda pendiente de clave.
Catálogos municipales	<code>bronze/municipios</code>	7 ciudades	Descubrimiento de portales municipales y preservación de catálogos raw.
Recursos municipales	<code>carpetas</code> <code>resources</code>	20 Madrid, 20 Sevilla	Prueba controlada de descarga para ampliar cobertura municipal.
Bronze inventariado	<code>inventario Bronze</code>	6.947.862 filas CSV	Evidencia raw, volumen, formatos y trazabilidad de archivos.
Silver limpio	<code>data_lake/silver</code>	182.149 filas en SQL	Datos curados y detalle operativo reutilizable.
Gold comparable	<code>indicators.csv</code>	3.975 indicadores	Indicadores homogéneos por ciudad, variable, fecha y fuente.
SQLite conversacional	<code>iseu_datos.sqlite</code>	93.561 filas	Tabla compatible con chatbot y consultas web.
Observaciones semánticas	<code>observaciones SQL</code>	105.391 filas	Capa intermedia para preguntas con más detalle territorial.

Nota. Los recursos nacionales OK suman INE, MITMA, SEPE y catálogos nacionales. AEMET se registra como `SKIPPED_AUTH` por requerir `AEMET_API_KEY`.

Tabla 6.6: Resumen de la capa Bronze inventariada

Métrica	Valor
Archivos totales inventariados	536
Filas CSV detectadas	6.947.862
Archivos en <code>bronze/apis</code>	82
Archivos municipales y catálogos	454
Ciudades municipales con catálogo	7
Recursos Madrid descargados en prueba	20
Recursos Sevilla descargados en prueba	20
Recursos Bilbao estructurados detectados	0

Nota. El inventario corresponde a la ejecución local registrada en `reports/inventory_bronze.json`.

sino como mayor disponibilidad de datos abiertos estructurados. Para evitar sesgos, el pipeline mantiene ese detalle en Bronze y Silver, pero agrega la información en Gold cuando se generan indicadores comparables.

Este punto es relevante para la defensa del proyecto como sistema Big Data. El volumen no se mide únicamente por el número de filas finales en SQL, sino por el conjunto del ciclo de vida del dato: recolección masiva, almacenamiento raw, inventario automático, limpieza, normalización, reducción analítica y carga estructurada. En la ejecución actual se detectaron casi 6,95 millones de filas CSV en Bronze. Ese volumen queda disponible para auditoría y reproceso, aunque la capa Gold final contenga un número menor de indicadores.

La estrategia evita confundir cantidad con calidad analítica. Por ejemplo, Barcelona aporta más de un millón de filas raw debido a la riqueza de su portal Open Data, mientras que otras ciudades ofrecen menos recursos estructurados. Si se cargaran todas esas filas como indicadores finales, cualquier análisis tendería a reflejar la abundancia documental de Barcelona más que una comparación urbana equilibrada. Por ello, el pipeline conserva el detalle en capas previas, pero calcula indicadores comparables para la capa de consulta.

Tabla 6.7: Archivos Bronze por fuente principal

Fuente	Archivos
Barcelona Open Data	255
Valencia Open Data	81
Zaragoza Open Data	73
SEPE	57
Madrid Datos Abiertos	21
Sevilla Open Data / ArcGIS Hub	21
Catálogos datos.gob.es	10
INE	9
MITMA	4
AEMET	2
Bilbao Open Data	2
Málaga Open Data	1

6.2.4. Capa Silver: datos limpios y seleccionados

La capa Silver no intenta limpiar todos los archivos descargados. La primera versión prioriza las fuentes con mayor utilidad para comparación multiciudad: INE para población, SEPE para empleo, MITMA para relación territorial y recursos municipales relevantes relacionados con renta, demografía, movilidad y sostenibilidad.

El resultado de Silver queda almacenado en `data_lake/silver/`. Esta capa mantiene más detalle que Gold y funciona como base intermedia para auditoría, futuras reglas de limpieza y ampliación del modelo. En la ejecución local se generaron siete tablas Silver.

La existencia de una tabla Silver municipal con 167.725 filas muestra que el sistema conserva detalle operativo. El incremento respecto a la ejecución anterior se debe principalmente a

Tabla 6.8: Tablas generadas en la capa Silver

Tabla Silver	Filas
ine/poblacion_municipal.csv	35
ine/poblacion_pais_nacimiento_total.csv	0
sepe/paro_registrado.csv	1.281
sepe/contratos_registrados.csv	1.242
sepe/demandantes_empleo.csv	1.372
mitma/relaciones_distrito_mitma.csv	10.494
municipios/municipal_prioritario.csv	167.725

la incorporación de recursos de Madrid, que aportan 6.450 filas de demografía municipal. Sevilla queda inventariada en Bronze, pero sus recursos ArcGIS requieren reglas específicas de lectura y normalización antes de incorporarse plenamente a Silver. Sin embargo, ese volumen no se traslada directamente a Gold, ya que hacerlo produciría un sesgo hacia las ciudades con más datos abiertos disponibles, especialmente Barcelona. Por ello, la capa Gold aplica agregaciones por ciudad, fecha y variable.

Silver cumple una función intermedia crítica. No es todavía la capa que consulta directamente el chat, pero tampoco es dato bruto. En esta zona se resuelven problemas técnicos como codificaciones distintas, separadores CSV diferentes, nombres de columnas heterogéneos, fechas en distintos formatos, valores numéricos escritos como texto y duplicados. También se filtran las ciudades objetivo para las fuentes nacionales, de modo que los datos de SEPE o INE puedan compararse con las fuentes municipales.

En la implementación actual, SEPE se filtra a las ciudades piloto para variables de empleo, INE se utiliza para población municipal, MITMA aporta relación territorial y los portales municipales aportan recursos prioritarios. Esta selección inicial permite obtener un pipeline funcional y defendible sin pretender resolver todos los datasets posibles en una sola iteración. El diseño, sin embargo, deja abierta la incorporación progresiva de nuevas reglas de limpieza.

La distribución municipal de Silver confirma que no todas las ciudades aportan el mismo tipo de información en esta fase. Barcelona concentra registros de renta con granularidad de barrio y sección censal; Valencia aporta movilidad y accidentes de tráfico; Zaragoza aporta movilidad y sostenibilidad; Madrid aporta demografía; y Sevilla queda pendiente de transformación específica desde servicios ArcGIS. Esta diferencia se documenta como una característica del ecosistema Open Data, no como un fallo del modelo: cada ayuntamiento publica con formatos, granularidades y mecanismos de acceso diferentes.

6.2.5. Capa Gold y base SQLite

La capa Gold se genera con un esquema universal de indicadores:

```
city, district, variable, value, date, source,
quality_score, category, unit
```

Este esquema permite que la página web y el chat consulten los datos de forma con-

sistente sin tener que conocer la estructura original de cada fuente. Por ejemplo, una pregunta sobre el paro en Valencia puede traducirse a una consulta SQL sobre la variable `unemployed_registered`. Una pregunta sobre contratos puede consultar `contracts_registered`, y una pregunta de población puede usar `population_total`.

La tabla Gold funciona como contrato de datos entre el pipeline y la capa conversacional. Mientras las fuentes originales pueden cambiar columnas, idiomas o formatos, la tabla Gold mantiene nombres estables. Esta estabilidad es necesaria para generar SQL desde lenguaje natural, porque el modelo debe conocer un vocabulario acotado de variables, categorías y unidades. Sin esta capa, cada pregunta obligaría al sistema a resolver de nuevo qué archivo abrir, qué columna usar y cómo interpretar el valor.

El uso de SQLite en local permite validar esta lógica sin desplegar todavía servicios cloud. SQLite no pretende ser la solución final de escalabilidad, sino una representación local del data warehouse. Permite probar consultas, verificar que el esquema funciona, alimentar la página web de desarrollo y revisar cómo respondería el chat ante preguntas reales. En AWS, esta misma capa se podría materializar en Redshift para consultas analíticas de alto rendimiento, o en Athena si se decide consultar directamente archivos en S3 mediante Glue Catalog.

La ejecución actual generó 3.975 indicadores Gold y los cargó en la base SQLite ubicada en `data_lake/gold/`

`iseu_indicadores.sqlite`. La tabla Gold principal se denomina `indicators`; además, se crea `indicator_catalog`, que resume variables, categorías, unidades y fuentes disponibles. No obstante, la base SQL ya no se limita a Gold: se amplió con una estructura enriquecida para conservar cantidad, detalle y capacidad de consulta conversacional.

La versión enriquecida de SQLite contiene cinco bloques principales. `indicators` conserva los indicadores Gold comparables. `indicadores` funciona como tabla compatible con el chatbot local y combina indicadores Gold con observaciones semantizadas de Silver. `semantic_observations` almacena observaciones normalizadas con ciudad, territorio, periodo, variable, valor, unidad, fuente, granularidad y calidad. Las tablas `silver_*` conservan el detalle limpio de Silver para auditoría y consultas avanzadas. Por último, `sql_table_catalog` documenta las tablas cargadas, su capa, número de filas, columnas y descripción funcional.

Tabla 6.9: Modelo SQLite enriquecido para consulta local y chatbot

Tabla	Filas	Función
<code>indicators</code>	3.975	Indicadores Gold comparables
<code>indicadores</code>	93.561	Tabla compatible con el chatbot local
<code>semantic_observations</code>	105.391	Observaciones Silver semantizadas
<code>silver_*</code>	182.149	Tablas Silver detalladas
<code>sql_table_catalog</code>	11	Catálogo de tablas SQL

Nota. La base enriquecida se copia también a `pag_web/Procesos/Datasets/iseu_datos.sqlite`, que es la ruta consumida por la página web local.

Figura 6.4

Consulta SQL sobre la tabla de indicadores Gold

```
●●● Consulta SQL sobre indicadores Gold

> sqlite3 data_lake/gold/iseu_indicadores.sqlite
sqlite> SELECT city, COUNT(*) AS indicadores
...> FROM indicators GROUP BY city ORDER BY indicadores DESC;

city      indicadores
-----
Valencia  586
Barcelona 579
Zaragoza  564
Malaga    562
Sevilla   562
Bilbao    561
Madrid    561

sqlite> SELECT variable, COUNT(*) FROM indicators GROUP BY variable;
job_seekers      1372
unemployed_registered 1281
contracts_registered 1242
population_total   35
income            15
income_median      15

Tablas: indicators, indicator_catalog, load_report
```

Nota. La consulta valida que la base SQLite concentra los indicadores finales por ciudad, variable, periodo, fuente y categoría.

Tabla 6.10: Indicadores Gold cargados en SQLite por ciudad

Ciudad	Indicadores
Valencia	586
Barcelona	579
Zaragoza	564
Málaga	562
Sevilla	562
Bilbao	561
Madrid	561

Nota. La distribución equilibrada se consigue agregando datos municipales en Gold, en lugar de cargar todos los registros detallados como indicadores finales.

Tabla 6.11: Indicadores Gold por variable principal

Variable	Filas
job_seekers	1.372
unemployed_registered	1.281
contracts_registered	1.242
population_total	35
income	15
income_median	15
income_per_household	6
income_per_person	6
mobility_resources_records	3

6.2.6. Relación con la página web y el chat

La lógica de consumo prevista es que la página web no consulte directamente los archivos CSV o JSON originales. En su lugar, el chat debe interpretar la pregunta del usuario, identificar ciudad, variable y periodo, y recuperar datos desde SQLite. Para comparaciones limpias entre ciudades puede utilizar `indicators`; para preguntas más abiertas o con mayor detalle territorial puede utilizar `indicadores`, `semantic_observations` o las tablas `silver_*`. De este modo, la respuesta se apoya en datos estructurados y trazables sin obligar al modelo a leer archivos raw.

Por ejemplo, una consulta como “¿Cómo ha evolucionado el paro en Valencia?” podría traducirse internamente a:

```
SELECT date, value
FROM indicators
WHERE city = 'Valencia'
      AND variable = 'unemployed_registered'
ORDER BY date;
```

Esta arquitectura separa tres responsabilidades: el data lake conserva los datos originales, el pipeline construye indicadores confiables y el modelo conversacional actúa como capa semántica para transformar lenguaje natural en SQL o en consultas controladas sobre SQLite. Para mejorar esta capa se implementó un diccionario semántico de variables en `pag_web/Procesos/semantic_dictionary.json`. Este diccionario no sustituye al modelo de lenguaje, sino que reduce la ambigüedad antes de llamar al modelo: términos como “paro”, “desempleo” o “personas sin trabajo” pueden apuntar a `unemployed_registered` o a la variable semántica “Paro registrado”; términos como “trabajo” u “oportunidades” pueden relacionarse con contratos registrados y demandantes de empleo; y expresiones como “renta”, “ingresos” o “poder adquisitivo” pueden mapearse a renta disponible, renta mediana o renta por hogar.

El diccionario semántico se integra en dos puntos del prototipo local. Primero, `sql_data.py` carga el archivo JSON, normaliza la pregunta del usuario y detecta intenciones, variables, categorías, fuentes, ciudades y necesidad de detalle territorial. Después, `server.py` incor-

Figura 6.5

Flujo de consulta del chat hacia SQL y respuesta web

```
●●● Flujo chat -> SQL -> respuesta

Usuario: Como ha evolucionado el paro en Valencia desde 2015?

Interpretacion semantica:
ciudad -> Valencia
variable -> unemployed_registered
periodo -> desde 2015

SQL generado y validado:
SELECT date, value
FROM indicators
WHERE city = 'Valencia'
      AND variable = 'unemployed_registered'
      AND date >= '2015-01-01'
ORDER BY date;

Respuesta web:
El sistema devuelve una serie temporal trazable desde SEPE,
lista para tabla, grafico o resumen textual.
```

Nota. El chat funciona como capa semántica: interpreta la pregunta, genera SQL controlado, consulta la tabla `indicators` y devuelve una respuesta trazable.

Tabla 6.12: Ejemplo de diccionario semántico propuesto para el chat

Términos de usuario	Variables o familias de datos sugeridas
paro, desempleo, sin trabajo	Paro registrado; demandantes de empleo
empleo, trabajo, oportunidades	Contratos registrados; demandantes de empleo
renta, ingresos, poder adquisitivo	Renta disponible; renta mediana; renta por hogar
accidentes, tráfico, siniestros	Accidentes de tráfico; movilidad urbana
población, habitantes, demografía	Población total; demografía municipal
tablas, fuentes, datos disponibles	sql_table_catalog; catálogo de variables

pora esa información al contexto enviado al modelo local, de forma que LM Studio recibe tanto los indicadores recuperados como la interpretación semántica usada para buscarlos.

Tabla 6.13: Componentes implementados del diccionario semántico SQL

Componente	Función
<code>semantic_dictionary.json</code>	Define alias, ciudades, intenciones, variables, categorías, fuentes y términos de comparación o detalle territorial.
<code>sql_data.py</code>	Expande la pregunta con términos semánticos, ajusta la puntuación de resultados y evita devolver indicadores aleatorios cuando la consulta pide tablas o fuentes.
<code>server.py</code>	Añade el resumen semántico al payload del chat y permite responder con el catálogo SQL cuando la pregunta trata sobre tablas disponibles.

La validación funcional del diccionario se realizó con preguntas equivalentes a lenguaje ciudadano. Por ejemplo, “personas sin trabajo” se interpretó como empleo y desempleo; “poder adquisitivo” se relacionó con renta; “siniestros de tráfico” se vinculó con accidentes; “bicicleta pública” se vinculó con movilidad; y “fuentes y tablas disponibles” se resolvió mediante el catálogo SQL, sin forzar la recuperación de indicadores numéricos que no correspondían a la pregunta.

En una implementación web, el chat no debería tener permisos libres sobre toda la base. La solución propuesta es que el modelo genere consultas dentro de un conjunto controlado de tablas y columnas. La aplicación puede disponer de una lista blanca de variables, fuentes y ciudades admitidas. Si la pregunta del usuario no puede mapearse a una variable disponible, el sistema debería responder que no cuenta con datos suficientes, en lugar de construir una consulta especulativa.

También es importante conservar la consulta SQL generada junto con la respuesta. Esto permitiría auditar qué dato se utilizó para contestar, repetir la consulta si se actualiza la base y detectar errores de interpretación. En un contexto académico y de análisis público, esta trazabilidad aporta valor porque permite explicar las respuestas del sistema, no solo generarlas.

La salida de la consulta puede transformarse en distintos formatos dentro de la web. Una pregunta de evolución temporal puede convertirse en un gráfico de líneas; una comparación entre ciudades puede mostrarse como tabla ordenada o gráfico de barras; y una pregunta de resumen puede devolver una explicación textual acompañada de la fuente y el periodo analizado. Por tanto, el SQL no es el resultado final visible para el usuario, sino el mecanismo interno que asegura que la respuesta del chat se base en datos reales.

6.2.7. Alcance SQL consolidado y limitaciones actuales

La decisión metodológica adoptada es mantener dos niveles SQL dentro de la misma base. El primer nivel es `indicators`, optimizado para preguntas frecuentes y comparaciones limpias

entre ciudades. El segundo nivel está formado por `indicadores`, `semantic_observations` y las tablas `silver_*`, que refuerzan el volumen analítico, conservan detalle operativo y permiten consultas más específicas. Esta solución evita presentar la reducción a 3.975 indicadores Gold como pérdida de datos, ya que el volumen limpio sigue disponible en SQLite.

Con esta estructura, el proyecto equilibra simplicidad y profundidad. El chat ciudadano puede priorizar `indicators` o `indicadores` para responder de forma clara, mientras que el análisis técnico puede consultar `semantic_observations` y las tablas `silver_*`. La tabla `sql_table_catalog` actúa como índice interno para que la aplicación conozca qué capas están disponibles, cuántas filas contienen y qué función cumple cada tabla.

La ejecución final deja el sistema en un estado suficiente para muestreo y demostración funcional. La capa Bronze contiene 6.947.862 filas CSV detectadas; Silver conserva 182.149 filas detalladas en SQL; y la tabla compatible con el chatbot contiene 93.561 filas consultables. Estos números permiten defender que ISEU+ trabaja con un ciclo de vida completo de datos, no solo con una pequeña tabla agregada.

No obstante, se mantienen limitaciones conocidas. AEMET requiere configurar `AEMET_API_KEY` para descargar datos meteorológicos reales. Bilbao queda como fuente parcial porque su portal no expone en esta fase un catálogo estructurado fácilmente explotable. Sevilla queda inventariada en Bronze mediante ArcGIS Hub, pero necesita reglas adicionales para transformar servicios *FeatureServer* y GeoJSON en indicadores Silver más ricos. La tabla de INE sobre población por país de nacimiento se descarga, pero el filtro actual devuelve cero filas, por lo que requiere revisión de esquema y valores de total. Finalmente, algunos recursos municipales presentan problemas de codificación de caracteres, visibles en nombres de barrios o campos con acentos; esto no invalida los valores numéricos, pero sí afecta a presentación, búsquedas textuales y calidad percibida.

Estas limitaciones no bloquean el paso a la conexión web con LM Studio. El conjunto de datos ya es suficiente para probar preguntas reales sobre empleo, población, renta, movilidad, accidentes, fuentes disponibles y cobertura SQL. Las mejoras pendientes se consideran iteraciones de calidad y ampliación, no requisitos previos para validar el funcionamiento del sistema.

Capítulo 7

Conclusiones

Contenido:

- Conclusiones del trabajo realizado.
- Puntos fuertes y puntos débiles del trabajo realizado.
- Limitaciones del trabajo realizado.
- Líneas de continuación del trabajo.

Referencias

Bibliografía

- [1] Referencia al documento 1.
- [2] Referencia al documento 2.
- [3] Última referencia añadida.